

RAPORT BEZPIECZEŃSTWA - APLIKACJA FLASK

Statyczna Analiza Kodu (SAST)

Przeprowadzono kompleksową analizę bezpieczeństwa aplikacji Flask. Poniżej przedstawiono zidentyfikowane podatności wraz z oceną ryzyka i rekomendacjami.

PODATNOŚĆ #1: DEBUG MODE w produkcji

Lokalizacja: main.py, linia 6

Krytyczność: KRYTYCZNA

Opis: Aplikacja uruchamiana z debug=True, co ujawnia szczegóły błędów i umożliwia wykonanie kodu.

Ryzyko: Atakujący może uzyskać dostęp do poufnych informacji i wykonać dowolny kod Python przez debugger.

Naprawa: Ustaw debug=False w produkcji. Użyj zmiennych środowiskowych do konfiguracji.

PODATNOŚĆ #2: Hardkodowany SECRET_KEY

Lokalizacja: src/init.py, linia 13

Krytyczność: KRYTYCZNA

Opis: SECRET_KEY ustawiony na 'secret' - łatwy do odgadnięcia.

Ryzyko: Możliwość podrobienia sesji użytkowników, przejęcia kont, ataków CSRF.

Naprawa: Użyj bezpiecznego, losowego klucza:

```
python
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')
# lub
app.config['SECRET_KEY'] = secrets.token_hex(32)
```

PODATNOŚĆ #3: Słaby algorytm hashowania (SHA-256)

Lokalizacja: src/auth.py, linia 35

Krytyczność: WYSOKA

Opis: Użycie przestarzałego method='sha256' bez salt. SHA-256 jest zbyt szybki - podatny

na ataki brute-force.

Ryzyko: Łamanie haseł metodą brute-force, rainbow tables.

Naprawa: Użyj pbkdf2:sha256 (domyślne) lub scrypt/argon2:

```
python
```

```
generate_password_hash(password1, method='pbkdf2:sha256')
```

PODATNOŚĆ #4: Duplikacja funkcji sign_up

Lokalizacja: src/auth.py, linia 14 i 51

Krytyczność: ŚREDNIA

Opis: Funkcja sign_up() zdefiniowana dwukrotnie - może powodować nieoczekiwane zachowanie.

Ryzyko: Błędy logiki, możliwość wykorzystania różnic między wersjami.

Naprawa: Usuń duplikat funkcji. Pozostaw jedną poprawną implementację.

PODATNOŚĆ #5: Niebezpieczne logowanie danych

Lokalizacja: src/init.py, linia 21

Krytyczność: WYSOKA

Opis: Logowanie całego request.get_data() może zapisywać hasła i dane wrażliwe w plaintext.

Ryzyko: Wrażliwe dane (hasła, tokeny) zapisane w logach.

Naprawa: Nie loguj request body lub filtruj wrażliwe pola. Użyj bezpiecznego systemu logowania.

PODATNOŚĆ #6: Słaba autoryzacja w /logs

Lokalizacja: src/auth.py, linia 82-83

Krytyczność: KRYTYCZNA

Opis: Sprawdzanie Location header zamiast adresu IP. Header można łatwo sfałszować.

Ryzyko: Każdy użytkownik może ustawić header 'Location: 127.0.0.1' i uzyskać dostęp do logów.

Naprawa: Użyj request.remote_addr lub @login_required z sprawdzeniem roli admina:

```
python
```

```
@auth.route('/logs', methods=['GET', 'POST'])
```

```
@login_required
```

```
def logs():
```

```
    if not current_user.is_admin:
```

```
        flash('Access denied', category='error')
```

```
return redirect(url_for('auth.login'))
# dalszy kod...
```

PODATNOŚĆ #7: Brak autoryzacji w delete_note

Lokalizacja: src/views.py, linia 48-58

Krytyczność: ŚREDNIA

Opis: Brak dekoratora @login_required - niezalogowani użytkownicy mogą próbować usuwać notatki.

Ryzyko: Próby nieautoryzowanego usuwania danych.

Naprawa: Dodaj @login_required przed funkcją delete_note():

```
python
@views.route('/delete-note', methods=['POST'])
@login_required
def delete_note():
    # kod...
```

PODATNOŚĆ #8: Słaba walidacja w /admin

Lokalizacja: src/admin.py, linia 11

Krytyczność: WYSOKA

Opis: Autoryzacja oparta tylko na email lub first_name - łatwo obejść.

Ryzyko: Każdy może utworzyć konto z first_name='admin' i uzyskać dostęp do panelu admina.

Naprawa: Dodaj pole 'is_admin' w modelu User:

```
python
# W models.py
class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(150), unique=True)
    password = db.Column(db.String(150))
    first_name = db.Column(db.String(150))
    is_admin = db.Column(db.Boolean, default=False)
    notes = db.relationship('Note')

# W admin.py
@admin.route('/admin', methods=['GET', 'POST'])
@login_required
def home():
    if current_user.is_admin:
        return render_template("admin.html", user=current_user)
    else:
        return redirect(url_for('auth.logout'))
```

PODATNOŚĆ #9: Podatność na Path Traversal

Lokalizacja: src/auth.py, linia 93-94

Krytyczność: KRYTYCZNA

Opis: Endpoint /<number> akceptuje dowolną wartość bez walidacji - może być wykorzystany do path traversal.

Ryzyko: Dostęp do dowolnych plików systemowych przez URL jak ../../etc/passwd

Naprawa: Usuń ten endpoint lub dodaj ścisłą walidację:

```
python
@auth.route('/<int:number>')
def input_number(number):
    return str(number)
```

PODATNOŚĆ #10: Brak CSRF Protection

Lokalizacja: Cała aplikacja

Krytyczność: WYSOKA

Opis: Brak Flask-WTF lub tokenów CSRF w formularzach.

Ryzyko: Ataki CSRF - wykonanie akcji w imieniu zalogowanego użytkownika.

Naprawa: Zainstaluj Flask-WTF i dodaj csrf_token do wszystkich formularzy:

```
bash
pip install Flask-WTF
python
# W __init__.py
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)

# W każdym szablonie HTML z formularzem
<form method="POST">
    {{ csrf_token() }}
    <!-- reszta formularza -->
</form>
```

PODATNOŚĆ #11: Słaba walidacja hasła

Lokalizacja: src/auth.py, linia 31

Krytyczność: ŚREDNIA

Opis: Wymagane tylko 7 znaków - zbyt krótkie hasło.

Ryzyko: Łatwe łamanie haseł.

Naprawa: Zwiększ minimum do 12 znaków, wymagaj cyfr, wielkich liter i znaków specjalnych:

```
python
import re

# Walidacja hasła
if len(password1) < 12:
    flash('Password must be at least 12 characters.', category='error')
elif not re.search(r'[A-Z]', password1):
    flash('Password must contain uppercase letter.', category='error')
elif not re.search(r'[a-z]', password1):
    flash('Password must contain lowercase letter.', category='error')
elif not re.search(r'[0-9]', password1):
    flash('Password must contain digit.', category='error')
elif not re.search(r'[!@#$%^&*(),.?":{}|<>]', password1):
    flash('Password must contain special character.', category='error')
```

PODATNOŚĆ #12: SQL Injection - brak parametryzacji

Lokalizacja: src/views.py

Krytyczność: NISKA

Opis: Kod używa ORM (SQLAlchemy), ale warto upewnić się, że nie ma surowych zapytań SQL.

Ryzyko: Potencjalna możliwość SQL Injection przy raw queries.

Naprawa: Kontynuuj używanie ORM. Jeśli używasz raw SQL, stosuj parametry wiązane:

```
python
# ŹŁE:
db.session.execute(f"SELECT * FROM users WHERE id = {user_id}")

# DOBRZE:
db.session.execute("SELECT * FROM users WHERE id = :id", {"id": user_id})
```

PODSUMOWANIE

Krytyczność	Liczba
KRYTYCZNA	4
WYSOKA	4
ŚREDNIA	3
NISKA	1

PRIORYTETOWE DZIAŁANIA:

1. **NATYCHMIAST** wyłączyć debug mode (debug=False)
2. **NATYCHMIAST** zmienić SECRET_KEY na bezpieczny losowy ciąg
3. Poprawić algorytm hashowania haseł na pbkdf2:sha256
4. Naprawić autoryzację w endpoint /logs
5. Usunąć lub zabezpieczyć endpoint /<number> (path traversal)
6. Dodać pole is_admin w modelu User i poprawić autoryzację admina
7. Wdrożyć Flask-WTF dla CSRF protection
8. Przestać logować request body z wrażliwymi danymi